Verilog coding

Day-01

To implement **all logic gates using all modeling styles in Verilog**, you need to understand the **three main modeling types**:

- **1. Gate-Level Modeling**
- **2. Dataflow Modeling**
- **3. Behavioral Modeling**

Below are examples of **basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)** using all three modeling styles.

All Logic Gates (Gate-Level Modeling)

```
module all_gates_gl(
   input a, b,
   output and_out,
   output or_out,
   output not_out,
   output nand_out,
   output nor_out,
   output xor_out,
   output xnor_out
);

  and  G1 (and_out, a, b);
  or   G2 (or_out, a, b);
  not  G3 (not_out, a);
  nand G4 (nand_out, a, b);
  nor  G5 (nor_out, a, b);
  xor  G6 (xor_out, a, b);
  xnor G7 (xnor_out, a, b);

endmodule
```

Dataflow Modeling

```
module all_gates_df(
   input a, b,
   output and_out,
   output or_out,
   output not_out,
   output nand_out,
   output nor_out,
   output xor_out,
   output xnor_out
```

```
);
  assign and_out  = a & b;
  assign or_out   = a | b;
  assign not_out  = ~a;
  assign nand_out = ~(a & b);
  assign nor_out  = ~(a | b);
  assign xor_out  = a ^ b;
  assign xnor_out = ~(a ^ b);

endmodule
```

Behavioral Modeling

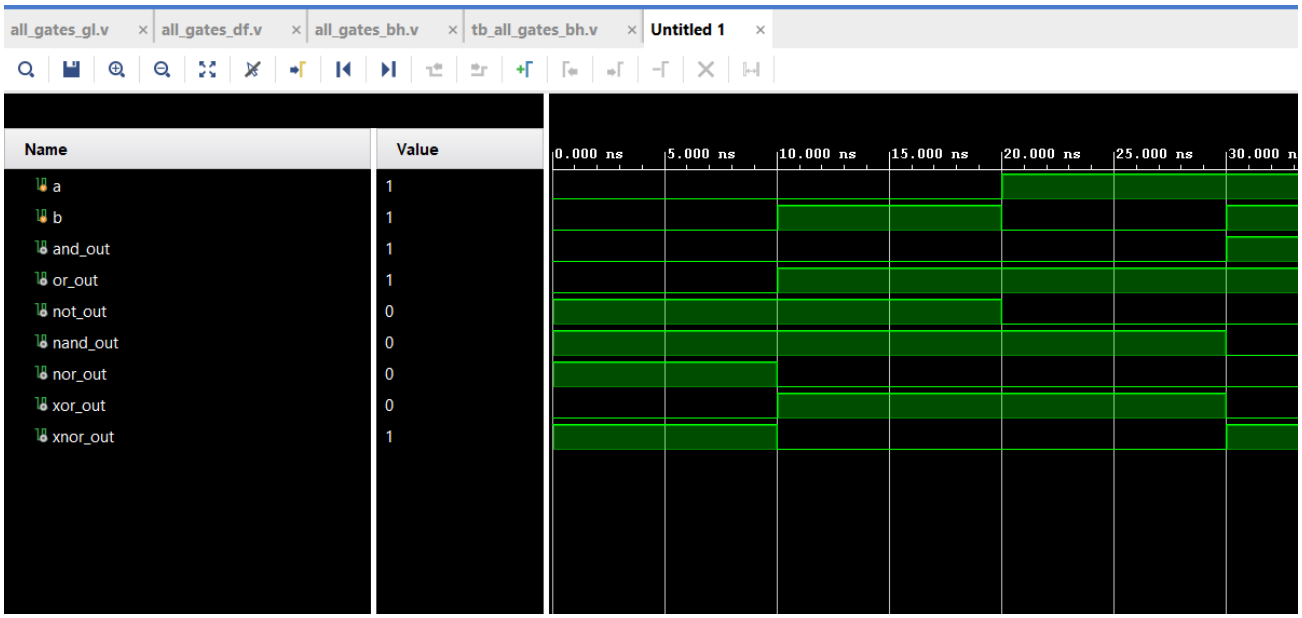
```
module all_gates_bh(
    input a, b,
    output reg and_out,
    output reg or_out,
    output reg not_out,
    output reg nand_out,
    output reg nor_out,
    output reg xor_out,
    output reg xnor_out
);
  always @(*) begin
    and_out  = a & b;
    or_out   = a | b;
    not_out  = ~a;
    nand_out = ~(a & b);
    nor_out  = ~(a | b);
    xor_out  = a ^ b;
    xnor_out = ~(a ^ b);
  end
endmodule
```

Test bench

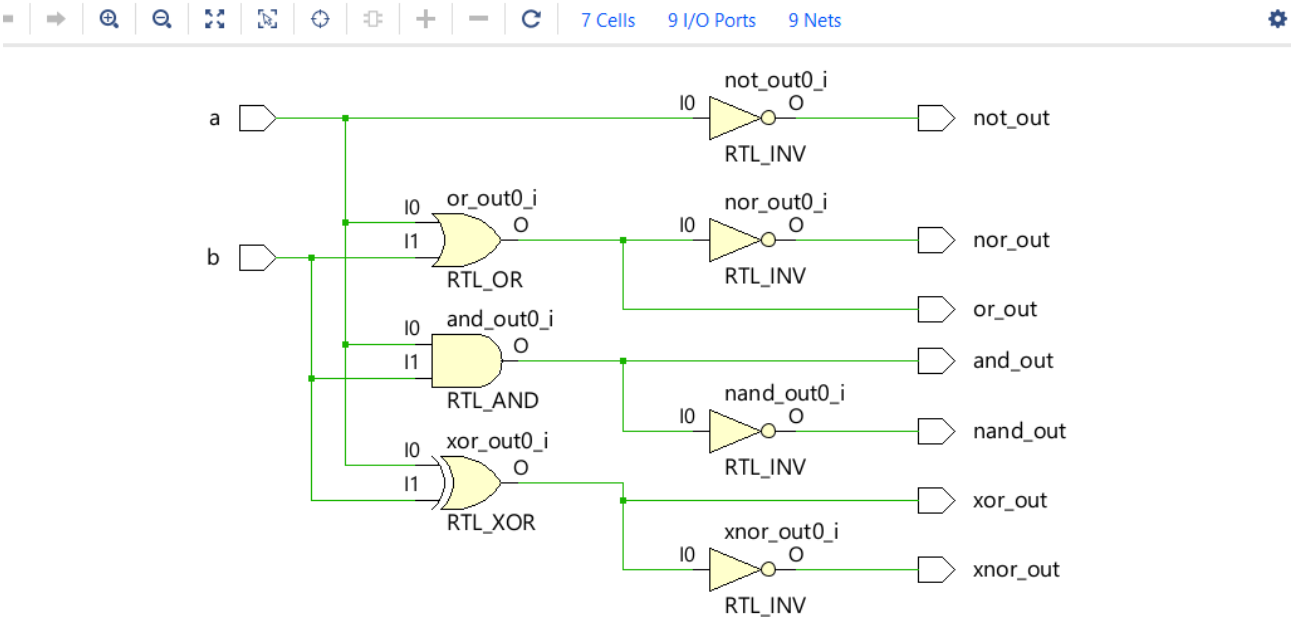
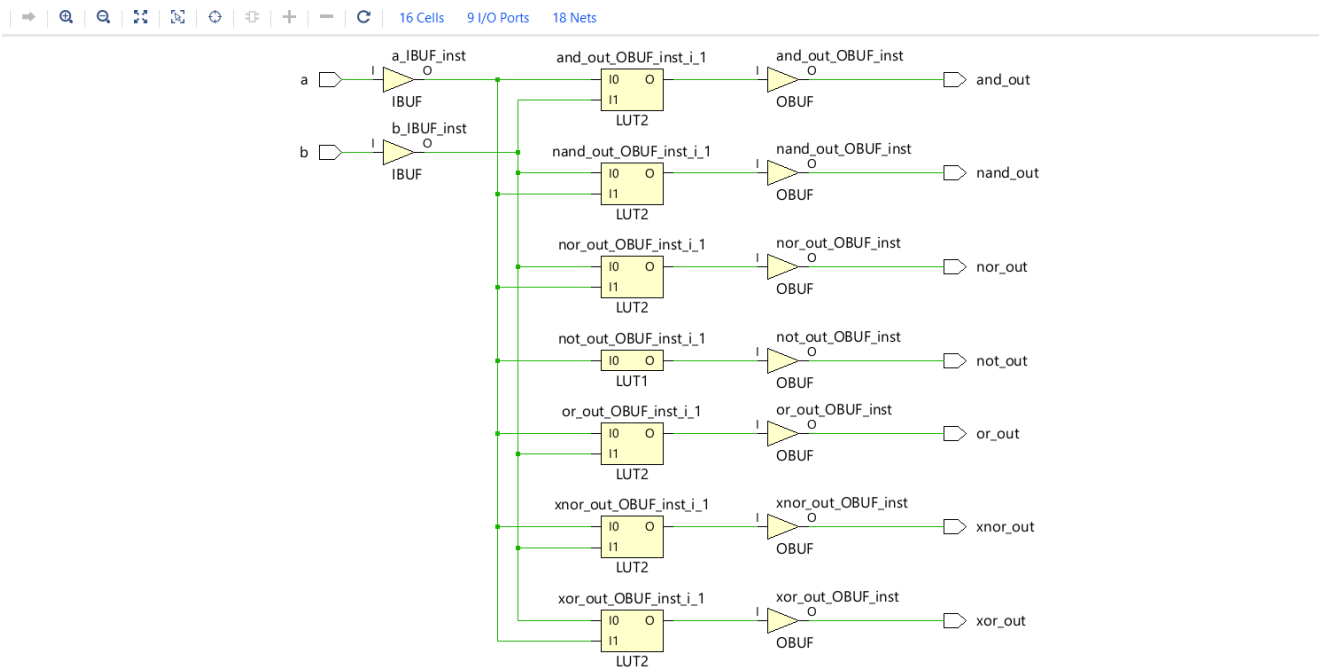
```
module tb_all_gates_bh;
  reg a, b;
  wire and_out, or_out, not_out;
  wire nand_out, nor_out, xor_out, xnor_out;
  all_gates_bh DUT (
    .a(a),
    .b(b),
    .and_out(and_out),
    .or_out(or_out),
```

```
    .not_out(not_out),
    .nand_out(nand_out),
    .nor_out(nor_out),
    .xor_out(xor_out),
    .xnor_out(xnor_out)
  );
  initial begin
    $monitor("%4t | %b %b |  %b   %b   %b    %b    %b   %b    %b",
            $time, a, b, and_out, or_out, not_out,
            nand_out, nor_out, xor_out, xnor_out);
    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;
    #10;
    $finish;
end
endmodule
```

waveform



RTL design



Schematic design